```
# Remove old versions
!pip uninstall -y transformers huggingface-hub tokenizers

# Install required versions
# Install compatible versions for this project
# Main Hugging Face Transformers library
# Download and manage models from Hugging Face Hub
# Optimizes training and inference on CPU/GPU
# Fast tokenizer library
!pip install \
transformers==4.46.3 \
huggingface_hub==0.26.2 \
accelerate==1.1.1 \
tokenizers==0.20.3 \
sentencepiece \
safetensors
```

```
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (3.29.7)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (26.2)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (2025.1
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (2.32.4)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (4.67.3)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub==0.26.2) (202
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub==0.2
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from accelerate==1.1.1) (5.9.5)
Requirement already satisfied: torch>=1.10.0 in /usr/local/lib/python3.12/dist-packages (from accelerate==1.1.1) (2.11.0+cu128
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.12/dist-packages (from torch>=1.10.0->accelerate==1.1.1
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.10.0->accelerate==1.1.1
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.10.0->accelerate==1.1
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.10.0->accelerate==1.1.1) (3.1.6
Requirement already satisfied: cuda-toolkit==12.8.1 in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart
Requirement already satisfied: cuda-bindings<13,>=12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.10.0->accele
Requirement already satisfied: nvidia-cudnn-cu12==9.19.0.56 in /usr/local/lib/python3.12/dist-packages (from torch>=1.10.0->acc
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.10.0->ac
Requirement already satisfied: nvidia-nccl-cu12==2.28.9 in /usr/local/lib/python3.12/dist-packages (from torch>=1.10.0->accele
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.10.0->acce
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.10.0->accelerate==1.1.1
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cub
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolk
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cub
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cub
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cu
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[c
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cubla
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->transformer
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->transformers==4.46.3) (3
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->transformers==4.46
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->transformers==4.46
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings<13,>=12.9.4
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1.10.0
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=1.10.0->accelera
Downloading transformers-4.46.3-py3-none-any.whl (10.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.0/10.0 MB 84.8 MB/s eta 0:00:00
Downloading huggingface_hub-0.26.2-py3-none-any.whl (447 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 447.5/447.5 kB 40.8 MB/s eta 0:00:00
Downloading accelerate-1.1.1-py3-none-any.whl (333 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 333.2/333.2 kB 33.2 MB/s eta 0:00:00
Downloading tokenizers-0.20.3-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (3.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.0/3.0 MB 79.4 MB/s eta 0:00:00
Installing collected packages: huggingface_hub, tokenizers, transformers, accelerate
  Attempting uninstall: accelerate
    Found existing installation: accelerate 1.14.0
    Uninstalling accelerate-1.14.0:
      Successfully uninstalled accelerate-1.14.0
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is th
diffusers 0.39.0 requires huggingface-hub<2.0,>=0.34.0, but you have huggingface-hub 0.26.2 which is incompatible.
gradio 6.20.0 requires huggingface-hub<2.0,>=1.2.0, but you have huggingface-hub 0.26.2 which is incompatible.
Successfully installed accelerate-1.1.1 huggingface_hub-0.26.2 tokenizers-0.20.3 transformers-4.46.3
```

```
# Import the PyTorch library for tensor operations and GPU support
import torch
```

```
# Import Hugging Face classes
# AutoModelForCausalLM = Automatically loads a causal language model (LLM)
# AutoTokenizer = Automatically loads the correct tokenizer for the model
# pipeline =Creates an easy-to-use inference pipeline
from transformers import AutoModelForCausalLM, AutoTokenizer, pipeline

# Load the pre-trained Phi-3 model
model = AutoModelForCausalLM.from_pretrained(

    "microsoft/Phi-3-mini-4k-instruct",

    # Automatically selects the available device (GPU/CPU)
    device_map="auto",

    # Automatically chooses the appropriate tensor datatype
    # (float16 on GPU, float32 on CPU)
    torch_dtype="auto",

    # Allows execution of custom model code provided by the model repository
    trust_remote_code=True,
)

# Load the tokenizer
tokenizer = AutoTokenizer.from_pretrained(

    # Load the tokenizer associated with the Phi-3 model
    "microsoft/Phi-3-mini-4k-instruct"
)

# Create a text generation pipeline
pipe = pipeline(

    # Specify the task
    "text-generation",

    # Use the previously loaded model
    model=model,

    # Use the loaded tokenizer
    tokenizer=tokenizer,

    # Return only the generated response (not the original prompt)
    return_full_text=False,

    # Maximum number of new tokens the model can generate
    max_new_tokens=500,

    # Disable random sampling for deterministic output
    # Same prompt = Same answer
    do_sample=False,
)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
config.json: 100%                                                 967/967 [00:00<00:00, 79.6kB/s]
configuration_phi3.py:       11.2k/? [00:00<00:00, 994kB/s]
A new version of the following files was downloaded from https://huggingface.co/microsoft/Phi-3-mini-4k-instruct:
- configuration_phi3.py
. Make sure to double-check they do not contain any added malicious code. To avoid downloading new versions of the code file, yo
modeling_phi3.py:       73.2k/? [00:00<00:00, 5.90MB/s]
A new version of the following files was downloaded from https://huggingface.co/microsoft/Phi-3-mini-4k-instruct:
- modeling_phi3.py
. Make sure to double-check they do not contain any added malicious code. To avoid downloading new versions of the code file, yo
WARNING:transformers_modules.microsoft.Phi-3-mini-4k-instruct.f39ac1d28e925b323eae81227eaba4464caced4e.modeling_phi3:`flash-atte
WARNING:transformers_modules.microsoft.Phi-3-mini-4k-instruct.f39ac1d28e925b323eae81227eaba4464caced4e.modeling_phi3:Current `fl
model.safetensors.index.json:       16.5k/? [00:00<00:00, 333kB/s]
Downloading shards: 100%                                          2/2 [01:13<00:00, 34.47s/it]
model-00001-of-00002.safetensors: 100%                                  4.97G/4.97G [00:49<00:00, 148MB/s]
model-00002-of-00002.safetensors: 100%                                  2.67G/2.67G [00:23<00:00, 97.0MB/s]
Loading checkpoint shards: 100%                                       2/2 [00:31<00:00, 14.90s/it]
generation_config.json: 100%                                        181/181 [00:00<00:00, 17.1kB/s]
tokenizer_config.json:       3.44k/? [00:00<00:00, 229kB/s]
tokenizer.model: 100%                                             500k/500k [00:00<00:00, 3.35MB/s]
tokenizer.json:       1.94M/? [00:00<00:00, 44.0MB/s]
added_tokens.json: 100%                                             306/306 [00:00<00:00, 30.8kB/s]
special_tokens_map.json: 100%                                         599/599 [00:00<00:00, 68.8kB/s]
```

```
# Create the prompt as a chat conversation
messages = [
    {
        "role": "user",                     # The sender of the message
        "content": "Create a funny joke about chickens."  # User's prompt
    }
]

# Generate a response from the Phi-3 model
output = pipe(messages)

# Print only the generated response
print(output[0]["generated_text"])
```

```
The `seen_tokens` attribute is deprecated and will be removed in v4.41. Use the `cache_position` model input instead.
`get_max_cache()` is deprecated for all Cache classes. Use `get_max_cache_shape()` instead. Calling `get_max_cache()` will raise
WARNING:transformers_modules.microsoft.Phi-3-mini-4k-instruct.f39ac1d28e925b323eae81227eaba4464caced4e.modeling_phi3:You are not
 Why did the chicken join the band? Because it had the drumsticks!
```

```
# Convert the chat messages into the model's expected prompt format
prompt = pipe.tokenizer.apply_chat_template(
    messages,          #Previously given messages role ,content,assistant
    tokenize=False     # Return the prompt as plain text instead of token IDs
)

# Display the formatted prompt
print(prompt)
```

```
<|user|>
Create a funny joke about chickens.<|end|>
<|endoftext|>
```

```
# Generate a response using random sampling
output = pipe(
    messages,          # previosuly given messages role,content,assistant
    do_sample=True,    # Enable random sampling instead of always choosing the highest-probability word
```

```
    temperature=1      # Set the randomness level (1 = moderate randomness)
)

# Print the generated response
print(output[0]["generated_text"])
```

```
 Why did the chicken apply for a job? To get to the coop-erate.
```

```
# Generate a response using nucleus (top-p) sampling
output = pipe(
    messages,          # Input conversation
    do_sample=True,    # Enable random sampling
    top_p=1            # Consider 100% of the probability distribution
)

# Print the generated response
print(output[0]["generated_text"])
```

```
 Why did the chicken join a band? Because it had the drumsticks!
```

```
# Define the AI's role/persona
persona = "You are an expert in Large Language models. You excel at breaking down complex papers into digestible summaries.\n"

# Tell the model what task to perform
instruction = "Summarize the key findings of the paper provided.\n"

# Provide additional context about the expected summary
context = "Your summary should extract the most crucial points that can help researchers quickly understand the most vital inform

# Specify the format of the output
data_format = "Create a bullet-point summary that outlines the method. Follow this up with a concise paragraph that encapsulates

# Specify the target audience
audience = "The summary is designed for busy researchers that quickly need to grasp the newest trends in Large Language Models.\n

# Specify the writing style/tone
tone = "The tone should be professional and clear.\n"

# Text that needs to be summarized
text = "MY TEXT TO SUMMARIZE"

# Combine the text with a label
data = f"Text to summarize: {text}"

# Build the complete prompt by combining all prompt components
query = (
    persona
    + instruction
    + context
    + data_format
    + audience
    + tone
    + data
)
```

```
# Import the required Hugging Face classes
from transformers import AutoModelForCausalLM, AutoTokenizer, pipeline

# Load the pre-trained Phi-3 language model
model = AutoModelForCausalLM.from_pretrained(
    "microsoft/Phi-3-mini-4k-instruct",
    device_map="auto",
    torch_dtype="auto",
    trust_remote_code=True
)

# Load the tokenizer for the Phi-3 model
tokenizer = AutoTokenizer.from_pretrained(
    "microsoft/Phi-3-mini-4k-instruct",
    trust_remote_code=True
)

# Create a text-generation pipeline
pipe = pipeline(
    "text-generation",
```

```
    model=model,
    tokenizer=tokenizer
)

# Create a one-shot prompt with one example
one_shot_prompt = [

    # User introduces a new word and asks for an example sentence
    {
        "role": "user",
        "content": "A 'Gigamuru' is a type of Japanese musical instrument. An example of a sentence that uses the word Gigamuru
    },

    # Assistant provides an example sentence
    {
        "role": "assistant",
        "content": "I have a Gigamuru that my uncle gave me as a gift. I love to play it at home."
    },

    # User introduces another new word and expects the model to generate a similar example
    {
        "role": "user",
        "content": "To 'screeg' something is to swing a sword at it. An example of a sentence that uses the word screeg is:"
    }
]

# Convert the chat messages into the model's required prompt format
print(
    tokenizer.apply_chat_template(
        one_shot_prompt,
        tokenize=False
    )
)
```

```
Loading checkpoint shards: 100%  2/2 [00:24<00:00, 11.23s/it]
WARNING:accelerate.big_modeling:Some parameters are on the meta device because they were offloaded to the cpu.
<|user|>
A 'Gigamuru' is a type of Japanese musical instrument. An example of a sentence that uses the word Gigamuru is:<|end|>
<|assistant|>
I have a Gigamuru that my uncle gave me as a gift. I love to play it at home.<|end|>
<|user|>
To 'screeg' something is to swing a sword at it. An example of a sentence that uses the word screeg is:<|end|>
<|endoftext|>
```

```
# Generate a response from the model using the one-shot prompt
outputs = pipe(
    one_shot_prompt,
    max_new_tokens=50
)

# Print the generated response
print(outputs[0]["generated_text"])
```

```
[{'role': 'user', 'content': "A 'Gigamuru' is a type of Japanese musical instrument. An example of a sentence that uses the word
```

```
# Create a user prompt asking the model to generate a product name and slogan
product_prompt = [
    {
        "role": "user",
        "content": "Create a name and slogan for a chatbot that leverages LLMs."
    }
]

# Generate a response from the model
outputs = pipe(
    product_prompt,
    max_new_tokens=100
)

# Extract the generated text from the model's output
product_description = outputs[0]["generated_text"]

# Display the generated product name and slogan
print(product_description)
```

```
[{'role': 'user', 'content': 'Create a name and slogan for a chatbot that leverages LLMs.'}, {'role': 'assistant', 'content': '
```

```
# Create a prompt asking the model to generate a sales pitch
# using the previously generated product description
sales_prompt = [
    {
        "role": "user",
        "content": f"Generate a very short sales pitch for the following product: '{product_description}'"
    }
]

# Generate a response from the model
outputs = pipe(
    sales_prompt,
    max_new_tokens=100
)

# Extract the generated sales pitch
sales_pitch = outputs[0]["generated_text"]

# Display the generated sales pitch
print(sales_pitch)
```

```
[{'role': 'user', 'content': 'Generate a very short sales pitch for the following product: \'[{\'role\': \'user\', \'content\':
```

```
# Create a Chain-of-Thought (CoT) prompt with one solved example
cot_prompt = [

    # prompt
    {
        "role": "user",
        "content": "Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis
    },

    # Assistant shows the reasoning step-by-step
    {
        "role": "assistant",
        "content": "Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11."
    },

    # prompt to solve using the same reasoning style
    {
        "role": "user",
        "content": "The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?
    }
]

# Generate the response from the model
outputs = pipe(
    cot_prompt,
    max_new_tokens=100
)

# Print the generated answer
print(outputs[0]["generated_text"])
```

```
[{'role': 'user', 'content': 'Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How ma
```

```
# Create a Zero-Shot Chain-of-Thought prompt
zeroshot_cot_prompt = [

    # Ask the model to solve the problem and explicitly
    {
        "role": "user",
        "content": "The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?
    }
]

# Generate a response from the model
outputs = pipe(
    zeroshot_cot_prompt,
    max_new_tokens=100
)
```

```
# Print the generated answer
print(outputs[0]["generated_text"])
```

```
[{'role': 'user', 'content': "The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do t
```

```
# Zero-shot tree-of-thought
zeroshot_tot_prompt = [{
    "role": "user",
    "content": """Imagine three different experts are answering this question. All experts will write down 1 step of their thin
}]
```

```
# Generate the output
outputs = pipe(zeroshot_tot_prompt,max_new_tokens=100)
print(outputs[0]["generated_text"])
```

```
[{'role': 'user', 'content': "Imagine three different experts are answering this question. All experts will write down 1 step of
```

```
# Zero-shot learning: Providing no examples
zeroshot_prompt = [
{"role": "user", "content": "Create a character profile for an RPG game in JSON format."}]
# Generate the output
outputs = pipe(zeroshot_prompt)
print(outputs[0]["generated_text"])
```

```
/usr/local/lib/python3.12/dist-packages/transformers/generation/utils.py:1375: UserWarning: Using the model-agnostic default `ma
  warnings.warn(
[{'role': 'user', 'content': 'Create a character profile for an RPG game in JSON format.'}, {'role': 'assistant', 'content': ' `
```

```
# One-shot learning: Providing an example of the output structure
one_shot_template = """Create a short character profile for an
RPG game. Make sure to only use this format:
{
"description": "A SHORT DESCRIPTION",
"name": "THE CHARACTER'S NAME",
"armor": "ONE PIECE OF ARMOR",
"weapon": "ONE OR MORE WEAPONS"
}
"""
one_shot_prompt = [{"role": "user", "content": one_shot_template}]
# Generate the output
outputs = pipe(one_shot_prompt,max_new_tokens=100)
print(outputs[0]["generated_text"])
```

```
[{'role': 'user', 'content': 'Create a short character profile for an\nRPG game. Make sure to only use this format:\n{\n"descrip
```

```
# Import the garbage collection module
# The garbage collector automatically removes objects that are no longer being used, helping to free RAM.
import gc

# Import the PyTorch library
import torch

# Delete the model, tokenizer, and pipeline objects from memory
del model, tokenizer, pipe

# Run Python's garbage collector to free unused memory
gc.collect()

# Clear the GPU cache used by PyTorch
torch.cuda.empty_cache()
```

```
# Install the llama-cpp-python library for running GGUF models locally
# This library allows  to run GGUF models locally without using the Hugging Face Transformers library.
# GGUF (GPT-Generated Unified Format) is a model file format designed for efficient local inference.
!pip install llama-cpp-python

# Import the Llama class from the llama_cpp library
from llama_cpp.llama import Llama

# Load the Phi-3 GGUF model
```

```
llm = Llama.from_pretrained(

    # Hugging Face repository containing the GGUF model
    repo_id="microsoft/Phi-3-mini-4k-instruct-gguf",

    # Load the FP16 GGUF model file
    filename="*fp16.gguf",

    # Use all available GPU layers for faster inference
    n_gpu_layers=-1,

    # Set the maximum context window to 2048 tokens
    n_ctx=2048,

    # Disable detailed loading logs
    verbose=False
)
```

```
Collecting llama-cpp-python
  Downloading llama_cpp_python-0.3.34.tar.gz (71.6 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 71.6/71.6 MB 11.4 MB/s eta 0:00:00
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Installing backend dependencies ... done
  Preparing metadata (pyproject.toml) ... done
Requirement already satisfied: typing-extensions>=4.5.0 in /usr/local/lib/python3.12/dist-packages (from llama-cpp-python) (4.16
Requirement already satisfied: numpy>=1.20.0 in /usr/local/lib/python3.12/dist-packages (from llama-cpp-python) (2.0.2)
Collecting diskcache>=5.6.1 (from llama-cpp-python)
  Downloading diskcache-5.6.3-py3-none-any.whl.metadata (20 kB)
Requirement already satisfied: jinja2>=2.11.3 in /usr/local/lib/python3.12/dist-packages (from llama-cpp-python) (3.1.6)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2>=2.11.3->llama-cpp-python
Downloading diskcache-5.6.3-py3-none-any.whl (45 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 45.5/45.5 kB 4.7 MB/s eta 0:00:00
Building wheels for collected packages: llama-cpp-python
  Building wheel for llama-cpp-python (pyproject.toml) ... done
  Created wheel for llama-cpp-python: filename=llama_cpp_python-0.3.34-py3-none-linux_x86_64.whl size=20581063 sha256=70466f1b17
  Stored in directory: /root/.cache/pip/wheels/4a/10/e7/0eb9b120f1640844f33562a3964c5b18b67de1d66d3f9530e8
Successfully built llama-cpp-python
Installing collected packages: diskcache, llama-cpp-python
Successfully installed diskcache-5.6.3 llama-cpp-python-0.3.34
Phi-3-mini-4k-instruct-fp16.gguf:  100%                                  7.64G/7.64G  [01:24<00:00,  60.1MB/s]
```

```
# Generate a chat completion from the loaded LLM
output = llm.create_chat_completion(

    # Provide the conversation (prompt) to the model
    messages=[
        {
            "role": "user",
            "content": "Create a warrior for an RPG in JSON format."
        },
    ],

    # Force the model to return the output as a valid JSON object
    response_format={"type": "json_object"},

    # Set temperature to 0 for deterministic and consistent output
    temperature=0,

# Extract only the generated JSON response from the model's output
)['choices'][0]['message']["content"]
```

```
# Import the built-in JSON library
import json

# Convert the JSON string into a Python dictionary and  then format it with proper indentation
json_output = json.dumps(
    json.loads(output),
    indent=4
)

# Display the formatted JSON
print(json_output)
```

```
{
    "warrior": {
```

```
            "name": "Eldric Stormbringer",
            "class": "Warrior",
            "level": 5,
            "attributes": {
                "strength": 18,
                "dexterity": 10,
                "constitution": 16,
                "intelligence": 8,
                "wisdom": 10,
                "charisma": 12
            },
            "skills": [
                {
                    "name": "Martial Arts",
                    "proficiency": 20,
                    "description": "Expert in hand-to-hand combat and weapon handling."
                },
                {
                    "name": "Shield Block",
                    "proficiency": 18,
                    "description": "Highly skilled at deflecting attacks with a shield."
                },
                {
                    "name": "Heavy Armor",
                    "proficiency": 16,
                    "description": "Expertly equipped with heavy armor for protection."
                },
                {
                    "name": "Survival",
                    "proficiency": 14,
                    "description": "Adept at finding food, water, and shelter in the wilderness."
                }
            ],
            "equipment": [
                {
                    "name": "Iron Sword",
                    "type": "Weapon",
                    "damage": 12,
                    "durability": 100
                },
                {
                    "name": "Chainmail Armor",
                    "type": "Armor",
                    "defense": 18,
                    "durability": 100
                },
                {
                    "name": "Leather Boots",
```